SRM Institute of Science and Technology
College of Engineering and Technology
School of Computing

Mode of Exam
OFFLINE

DEPARTMENT OF COMPUTING TECHNOLOGIES

SRM Nagar, Kattankulathur – 603203, Chengalpattu District, Tamilnadu

Test: CLAT-1 **Date:**
Course Code & Title: 21CSC203P & Advamnced Programming Practice **Duration:** 2 hrs
Year & Sem: II & III **ax. Marks:** 50

Part - A
(5 x 4 = 20 Marks)
Instructions: Answer any 5

Q. No	Question	Marks	BL	CO	PO	PI Code

1	Compare and contrast Imperative programming and Declarative Programming.
	Answer :
	Imperative Programming:
	Imperative programming focuses on how to perform tasks by specifying the step-by-step instructions that change the program's state. The programmer explicitly defines control flow, loops, and the sequence of operations.
	
	Example
	
	int sum = 0;
	for (int i = 1; i <= 5; i++) {
	sum += i;
	}
	System.out.println(sum); // Output: 15
	
	Declarative Programming:
	Declarative programming focuses on what needs to be done, rather than how to do it. The programmer defines the desired outcome or result without detailing the control flow or steps to achieve it. The implementation is handled by the underlying system or language.
	
	Example
	
	SELECT SUM(number) FROM numbers WHERE number <= 5;
		4	**2**	**1**	**2**	**1.6.1***

2	What is data encapsulation, and why is it useful? **Answer** : Encapsulation is a concept in Object-Oriented Programming for combining properties and methods in a single unit. Encapsulation helps developers follow a modular approach for software development because each object has its own set of methods and variables and serves its functions independent of other objects. In addition to that, encapsulation serves data hiding purposes.	**4**	**1**	**1**	**2**	**1.6.1**
3	Write is the output of following code class Test { public static void main (String args[])	**4**	**2**	**2**	**2**	**1.6.1**

	{
	System.out.println(10 + 20 + "Javatpoint");
	System.out.println("Javatpoint" + 10 + 20);
	}
	}
	Answer
	30Javatpoint
	Javatpoint1020					

4	Differentiate Concurrent and Parallel programming with example. **Answer :** **Concurrent Programming:** Refers to the concept where multiple tasks or processes progress simultaneously in overlapping time periods. These tasks may not actually be running at the same time but are managed in such a way that they appear to be executing concurrently by interleaving steps. The focus is on task structure and responsiveness, even if not all tasks run simultaneously. **Parallel Programming:** In parallel programming, multiple tasks are executed simultaneously across multiple cores or processors. The tasks truly run at the same time. The focus is on dividing a large task into smaller subtasks that can be executed simultaneously for better performance.	**4**	**2**	**1**	**2**	**1.6.1**
5	Discuss about Böhm-Jacopini theorem with suitable example? It states that any algorithm can be written using only three basic control structures: Sequence: Executing statements one after another. Selection: Making decisions, typically represented by if-else statements or switch cases. Iteration: Repeating tasks, often represented by while, for, or do-while loops.	**4**	**2**	**1**	**2**	**1.6.1**

6	Demonstrate constructors in java with suitable example.
	class Person {
	// Attributes
	String name;
	int age;
	
	// Default constructor (no arguments)
	public Person() {
	name = "Unknown";
	age = 0;
	System.out.println("Default constructor called");
	}
	
	// Parameterized constructor
	public Person(String name, int age) {
	this.name = name;
	this.age = age;
	System.out.println("Parameterized constructor called");
		4	**3**	**2**	**2**	**1.6.1**

	}
	
	// Method to display person's details
	public void displayInfo() {
	System.out.println("Name: " + name + ", Age: " + age);
	}
	}
	
	public class ConstructorDemo {
	public static void main(String[] args) {
	// Creating an object using the default constructor
	Person person1 = new Person();
	person1.displayInfo(); // Output: Name: Unknown, Age: 0
	
	// Creating an object using the parameterized constructor
	Person person2 = new Person("Alice", 25);
	person2.displayInfo(); // Output: Name: Alice, Age: 25
	}
	}					

Part – B **(3 x 10 = 30 Marks)** **Instructions: Answer any 3**						

7	A newly constructed University is going to manage book details in the library where each book has a title, author, ISBN number, and the number of copies available. They are in the need of computer software which is going to manage libray for easy management of book. You are requested to design a a Book class with fields for title, author, ISBN, and copiesAvailable. Create a constructor to initialize these fields. Write a method displayBookInfo() to display the book's detail and also write a method addbookToLibrary()to additon of books.
	
	Answer :
	
	public class Book {
	
	// Fields for the book class
	
	private String title;
	
	private String author;
	
	private String ISBN;
	
	private int copiesAvailable;
	
	// Constructor to initialize book details
	
	public Book(String title, String author, String ISBN, int copiesAvailable) {
	
	this.title = title;
	
	this.author = author;
	
	this.ISBN = ISBN;
	
	this.copiesAvailable = copiesAvailable;
		10	**3**	**2**	**2**	**1.6.1**

```
    }


    // Method to display book details
    public void displayBookInfo() {
        System.out.println("Title: " + title);
        System.out.println("Author: " + author);
        System.out.println("ISBN: " + ISBN);
        System.out.println("Copies Available: " + copiesAvailable);
    }


    // Method to add more copies of the book to the library
    public void addBookToLibrary(int newCopies) {
        if (newCopies > 0) {
            copiesAvailable += newCopies;
            System.out.println(newCopies + " copies have been added. Total copies now: " + copiesAvailable);
        } else {
            System.out.println("Invalid number of copies to add.");
        }
    }


    // Main method to test the functionality
    public static void main(String[] args) {
        // Create a new book object
        Book book1 = new Book("The Catcher in the Rye", "J.D. Salinger", "978-0-316-76948-0", 5);


        // Display book information
        book1.displayBookInfo();


        // Add more copies of the book
```

	book1.addBookToLibrary(3);
	
	// Display updated book information
	book1.displayBookInfo();
	}
	}					

8	A company has different types of employees such as full-time and part-time employees. Full-time employees have a fixed salary, while part-time employees are paid by the hour. Create a superclass Employee with fields name, employeeId, and department. Create two subclasses: FullTimeEmployee (with a field fixedSalary) and PartTimeEmployee (with fields hourlyWage and hoursWorked). Implement methods in the subclasses to calculate the salary for each type of employee. Create a method in the Employee superclass called displayEmployeeDetails() and override it in both subclasses to display specific details for full-time and part-time employees. Discuss how method overriding works in inheritance.
	
	Answer
	
	// Superclass: Employee
	public class Employee {
	// Common fields for all employees
	protected String name;
	protected int employeeId;
	protected String department;
	// Constructor for Employee
	public Employee(String name, int employeeId, String department) {
	this.name = name;
	this.employeeId = employeeId;
	this.department = department;
	}
	// Method to display employee details (to be overridden)
	public void displayEmployeeDetails() {
		10	**3**	**2**	**3**	**1.6.1**

```
        System.out.println("Name: " + name);
        System.out.println("Employee ID: " + employeeId);
        System.out.println("Department: " + department);
    }
    // Abstract method to calculate salary (must be implemented in subclasses)
    public double calculateSalary() {
        return 0.0; // To be overridden
    }
}
// Subclass: FullTimeEmployee
class FullTimeEmployee extends Employee {
    // Field specific to full-time employees
    private double fixedSalary;
    // Constructor for FullTimeEmployee
    public FullTimeEmployee(String name, int employeeId, String department, double fixedSalary) {
        super(name, employeeId, department);  // Call to superclass constructor
        this.fixedSalary = fixedSalary;
    }
    // Override the calculateSalary method for full-time employees
    @Override
    public double calculateSalary() {
        return fixedSalary;
    }
    // Override the displayEmployeeDetails method
    @Override
    public void displayEmployeeDetails() {
```

```
        super.displayEmployeeDetails(); // Call the superclass method to display common details

        System.out.println("Employment Type: Full-time");

        System.out.println("Fixed Salary: $" + fixedSalary);

    }

}

// Subclass: PartTimeEmployee

class PartTimeEmployee extends Employee {

    // Fields specific to part-time employees

    private double hourlyWage;

    private int hoursWorked;



    // Constructor for PartTimeEmployee

    public PartTimeEmployee(String name, int employeeId, String department, double hourlyWage, int hoursWorked) {

        super(name, employeeId, department);  // Call to superclass constructor

        this.hourlyWage = hourlyWage;

        this.hoursWorked = hoursWorked;

    }

    // Override the calculateSalary method for part-time employees

    @Override

    public double calculateSalary() {

        return hourlyWage * hoursWorked;

    }

    // Override the displayEmployeeDetails method

    @Override

    public void displayEmployeeDetails() {

        super.displayEmployeeDetails(); // Call the superclass method to display common details
```

	System.out.println("Employment Type: Part-time");
	
	System.out.println("Hourly Wage: $" + hourlyWage);
	
	System.out.println("Hours Worked: " + hoursWorked);
	
	System.out.println("Calculated Salary: $" + calculateSalary());
	
	}
	
	}
	
	// Main class to test the implementation
	
	public class Main {
	
	public static void main(String[] args) {
	
	// Creating a FullTimeEmployee object
	
	FullTimeEmployee fullTimeEmp = new FullTimeEmployee("Alice", 101, "Engineering", 5000.00);
	
	// Display details and calculate salary for full-time employee
	
	fullTimeEmp.displayEmployeeDetails();
	
	System.out.println("Salary: $" + fullTimeEmp.calculateSalary());
	
	System.out.println("-------------------------");
	
	// Creating a PartTimeEmployee object
	
	PartTimeEmployee partTimeEmp = new PartTimeEmployee("Bob", 102, "Marketing", 20.00, 120);
	
	// Display details and calculate salary for part-time employee
	
	partTimeEmp.displayEmployeeDetails();
	
	System.out.println("Salary: $" + partTimeEmp.calculateSalary());
	
	}
	
	}					

9	A vehicle rental company offers various types of vehicles, such as cars, trucks, and bikes. Each vehicle type has a different way of calculating rental costs. Create an interface Rentable with methods rent() and calculateRentalCost(int days). Implement this interface in classes Car, Truck, and Bike, where each class provides its own method of calculating rental costs. Create an	**10**	**3**	**2**	**3**	**1.6.1**

abstract class Vehicle that implements the Rentable interface and provides common properties like make, model, and year. Extend this abstract class in the Car, Truck, and Bike classes. Discuss how abstraction allows for code reuse.

Answers

```
// Rentable interface

interface Rentable {

    void rent();  // Rent a vehicle

    double calculateRentalCost(int days);  // Calculate rental cost based on the number of days

}


// Abstract class Vehicle implements Rentable

abstract class Vehicle implements Rentable {

    // Common properties for all vehicles

    protected String make;

    protected String model;

    protected int year;


    // Constructor for Vehicle

    public Vehicle(String make, String model, int year) {

        this.make = make;

        this.model = model;

        this.year = year;

    }


    // Display common vehicle details

    public void displayDetails() {

        System.out.println("Make: " + make);

        System.out.println("Model: " + model);

        System.out.println("Year: " + year);

    }

}
```

```
// Car class extends Vehicle and implements Rentable
class Car extends Vehicle {
    private double dailyRate;

    public Car(String make, String model, int year, double dailyRate) {
        super(make, model, year);
        this.dailyRate = dailyRate;
    }

    // Implement the rent method
    @Override
    public void rent() {
        System.out.println("Car rented: " + make + " " + model);
    }

    // Implement calculateRentalCost method
    @Override
    public double calculateRentalCost(int days) {
        return days * dailyRate;
    }

    // Override displayDetails to show additional information
    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Daily Rental Rate: $" + dailyRate);
    }
}

// Truck class extends Vehicle and implements Rentable
```

```
class Truck extends Vehicle {
    private double dailyRate;
    private double cargoWeightLimit;

    public Truck(String make, String model, int year, double dailyRate, double cargoWeightLimit) {
        super(make, model, year);
        this.dailyRate = dailyRate;
        this.cargoWeightLimit = cargoWeightLimit;
    }

    // Implement the rent method
    @Override
    public void rent() {
        System.out.println("Truck rented: " + make + " " + model);
    }

    // Implement calculateRentalCost method
    @Override
    public double calculateRentalCost(int days) {
        return days * dailyRate;
    }

    // Override displayDetails to show additional information
    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Daily Rental Rate: $" + dailyRate);
        System.out.println("Cargo Weight Limit: " + cargoWeightLimit + " tons");
    }
}
```

```
// Bike class extends Vehicle and implements Rentable
class Bike extends Vehicle {
    private double hourlyRate;

    public Bike(String make, String model, int year, double hourlyRate) {
        super(make, model, year);
        this.hourlyRate = hourlyRate;
    }

    // Implement the rent method
    @Override
    public void rent() {
        System.out.println("Bike rented: " + make + " " + model);
    }

    // Implement calculateRentalCost method
    @Override
    public double calculateRentalCost(int days) {
        // Assuming 24 hours per day
        return days * 24 * hourlyRate;
    }

    // Override displayDetails to show additional information
    @Override
    public void displayDetails() {
        super.displayDetails();
        System.out.println("Hourly Rental Rate: $" + hourlyRate);
    }
}
```

	// Main class to test the implementation
	public class Main {
	public static void main(String[] args) {
	// Create a Car object and rent it for 5 days
	Car car = new Car("Toyota", "Camry", 2020, 50.0);
	car.displayDetails();
	car.rent();
	System.out.println("Rental cost for 5 days: $" + car.calculateRentalCost(5));
	System.out.println("----------");
	// Create a Truck object and rent it for 3 days
	Truck truck = new Truck("Ford", "F-150", 2019, 100.0, 5.0);
	truck.displayDetails();
	truck.rent();
	System.out.println("Rental cost for 3 days: $" + truck.calculateRentalCost(3));
	System.out.println("----------");
	// Create a Bike object and rent it for 2 days
	Bike bike = new Bike("Yamaha", "YZF-R3", 2021, 2.5);
	bike.displayDetails();
	bike.rent();
	System.out.println("Rental cost for 2 days: $" + bike.calculateRentalCost(2));
	}
	}					

10	You are working on a zoo simulation program. There are different animals such as Lion, Elephant, and Bird. Each animal can makeSound(), but the sound they make is different for each animal.You are requested to create a base class Animal with a	**10**	**3**	**2**	**3**	**1.6.1**

method makeSound().Implement subclasses Lion, Elephant, and Bird, overriding the makeSound() method in each.
Write a program that demonstrates how polymorphism can be used to make each animal produce its respective sound by calling makeSound() on an Animal reference.

Answer :

```
// Base class Animal

class Animal {

    // Method to be overridden by subclasses

    public void makeSound() {

        System.out.println("Some generic animal sound");

    }

}



// Subclass Lion

class Lion extends Animal {

    @Override

    public void makeSound() {

        System.out.println("Lion roars: Roar!");

    }

}



// Subclass Elephant

class Elephant extends Animal {

    @Override

    public void makeSound() {

        System.out.println("Elephant trumpets: Paaa!");

    }

}



// Subclass Bird

class Bird extends Animal {

    @Override

    public void makeSound() {
```

```
        System.out.println("Bird chirps: Chirp Chirp!");
    }
}


// Main class to demonstrate polymorphism
public class ZooSimulation {
    public static void main(String[] args) {
        // Creating instances of different animals
        Animal lion = new Lion();
        Animal elephant = new Elephant();
        Animal bird = new Bird();

        // Polymorphism in action: Each animal makes its own sound
        makeAnimalSound(lion);
        makeAnimalSound(elephant);
        makeAnimalSound(bird);
    }

    // Method to call makeSound() on an Animal reference
    public static void makeAnimalSound(Animal animal) {
        animal.makeSound();
    }
}
```

Question Paper Setter

Approved by Audit Professor/ Course Coordinator